

■ Thread Synchronization

Beginner Exam Cheat Sheet • Deadlocks • RW Locks • Barriers

■ 1. DEADLOCK

What is Deadlock?

A **deadlock** is a situation where a group of threads (or processes) are all stuck waiting for each other — and nobody can ever make progress. Think of it like two people in a narrow hallway, each waiting for the other to step aside.

Classic Example: Two Mutexes

Thread 1

```
lock(A); // gets A

lock(B); // WAITS for B

// ... work ...

unlock(B);

unlock(A);
```

Thread 2

```
lock(B); // gets B

lock(A); // WAITS for A

// ... work ...

unlock(A);

unlock(B);
```

◆ ■ Deadlock!

Thread 1 holds A and waits for B. Thread 2 holds B and waits for A. Neither can proceed → both wait forever.

How to Prevent Deadlock

Solution 1: Fixed Lock Ordering

Make ALL threads always lock mutexes in the SAME order (e.g., always lock A before B). This breaks the circular wait.

```
// BOTH threads must do this same order:

lock(A); // always lock A first

lock(B); // then B

// ... work ...

unlock(B);

unlock(A);
```

Solution 2: Try-and-Back-Off

Use a non-blocking trylock. If you can't get all locks, release what you have and try again.

```
while (true) {

    lock(A);
```

```

if (trylock(B) == SUCCESS) break; // got both!

unlock(A); // release A and retry

}

// ... do work with A and B ...

unlock(B); unlock(A);

```

◆ ■ Exam Tip

If a question shows threads locking mutexes in different orders, that's a deadlock risk. If all threads use the same order, there is no deadlock.

■ 2. READERS-WRITERS PROBLEM

The Problem

A shared data structure is read often but written rarely. We want to allow as much parallelism as possible while keeping data consistent.

Scenario	Allowed?
Multiple readers reading at the same time	■ YES
A reader starts while other readers are active	■ YES
Two writers writing at the same time	■ NO
A writer and a reader at the same time	■ NO
A reader starting while a writer is active	■ WAIT
A writer starting while readers/writers are active	■ WAIT

■ 3. PTHREAD READ-WRITE LOCKS (API)

The 5 Functions You Need to Know

Function	What it does
<code>pthread_rwlock_init(&rwlock, NULL)</code>	Initialize the rw-lock before using it
<code>pthread_rwlock_destroy(&rwlock;)</code>	Clean up / free the rw-lock when done
<code>pthread_rwlock_rdlock(&rwlock;)</code>	Acquire a READ lock (blocks if a writer holds it or is waiting)
<code>pthread_rwlock_wrlock(&rwlock;)</code>	Acquire a WRITE lock (blocks if ANY reader or writer holds it)
<code>pthread_rwlock_unlock(&rwlock;)</code>	Release the lock (same function for both read and write)

Complete Working Example

```
#include
```

```

#include

pthread_rwlock_t rwl; // declare the rw-lock (global)

int shared_data = 0; // data protected by the lock

void* reader(void* arg) {

pthread_rwlock_rdlock(&rwl); // acquire READ lock

printf("Read: %d\n", shared_data); // safe to read

pthread_rwlock_unlock(&rwl); // release

return NULL;

}

void* writer(void* arg) {

pthread_rwlock_wrlock(&rwl); // acquire WRITE lock

shared_data++; // safe to write

pthread_rwlock_unlock(&rwl); // release

return NULL;

}

int main() {

pthread_rwlock_init(&rwl;, NULL); // MUST initialize first

// ... create reader/writer threads ...

pthread_rwlock_destroy(&rwl); // cleanup at end

}

```

Key Behaviors of Pthread RW Lock

rdlock behavior:

- Thread gets the lock if **no writer holds it AND no writer is waiting**
- Multiple readers can hold it simultaneously
- Blocks (waits) if a writer holds it or is blocked on the lock

✦ Writer Priority

Pthread rwlock **prefers writers**: a new reader will be blocked if any writer is waiting, even if no writer is currently writing.

wrlock behavior:

- Thread gets the lock only if **NO reader and NO writer holds it**
- Only ONE writer allowed at a time
- Must also wait for active readers to finish

✦ Common Mistake

Forgetting to call `pthread_rwlock_unlock()` — always unlock no matter what path your code takes (use a single exit point).

■ 4. STARVATION (RW Lock Issues)

Starvation = a thread (or group) is perpetually denied access, even though the system is making progress overall. Different from deadlock!

Type	Cause	Effect
Writer Starvation	Too many readers keep arriving; writers never get a turn	Writers wait forever while readers continuously hold the lock
Reader Starvation	Too many writers keep arriving; readers never get a turn	Readers wait forever while writers continuously hold the lock

◆ ■ Exam Tip: Deadlock vs Starvation

Deadlock: Nobody makes progress — system is stuck completely.

Starvation: Some threads make progress, but certain threads (readers or writers) never get a turn. The system is not stuck, but it is unfair.

■ 5. BUILDING RW LOCKS WITH MUTEX + CONDITION VARIABLES

You can build a custom RW lock using a mutex and two condition variables. This gives you full control over reader/writer priority.

State Variables to Track

```
pthread_mutex_t mutex; // protects all state variables

pthread_cond_t read_cv; // readers wait on this

pthread_cond_t write_cv; // writers wait on this

int r_active = 0; // # readers currently reading

int w_active = 0; // # writers currently writing (0 or 1)

int r_wait = 0; // # readers waiting for the lock

int w_wait = 0; // # writers waiting for the lock
```

Pseudocode: rwlock_rdlock()

```
pthread_mutex_lock(&mutex);

r_wait++; // I am now waiting

while (w_active > 0 || w_wait > 0) // predicate: writer present?

pthread_cond_wait(&read_cv, &mutex); // sleep until notified

r_wait--; // done waiting

r_active++; // now actively reading
```

```
pthread_mutex_unlock(&mutex); // release mutex (NOT rwlock!)
```

```
// ---- do read operations here ----
```

Pseudocode: `rwlock_wrlock()`

```
pthread_mutex_lock(&mutex);
```

```
w_wait++; // I am now waiting
```

```
while (r_active > 0 || w_active > 0) // predicate: anyone active?
```

```
pthread_cond_wait(&write_cv, &mutex);
```

```
w_wait--; // done waiting
```

```
w_active++; // now actively writing
```

```
pthread_mutex_unlock(&mutex); // release mutex
```

```
// ---- do write operations here ----
```

Pseudocode: `rwlock_rdunlock()`

```
pthread_mutex_lock(&mutex);
```

```
r_active--; // I am done reading
```

```
if (w_wait > 0 && r_active == 0) // writer waiting & no more readers
```

```
pthread_cond_signal(&write_cv); // wake one writer
```

```
pthread_mutex_unlock(&mutex);
```

Pseudocode: `rwlock_wrunlock()`

```
pthread_mutex_lock(&mutex);
```

```
w_active--; // I am done writing
```

```
if (r_wait > 0) { // POLICY: prefer readers
```

```
pthread_cond_broadcast(&read_cv); // wake ALL waiting readers
```

```
} else if (w_wait > 0) { // else wake one writer
```

```
pthread_cond_signal(&write_cv);
```

```
}
```

```
pthread_mutex_unlock(&mutex);
```

The **while loop predicate** is the condition that blocks a thread. It answers: "Should I wait?" If TRUE → keep waiting. If FALSE → proceed.

For readers: **while (w_active > 0 || w_wait > 0)** — wait if a writer is active OR waiting.

For writers: **while (r_active > 0 || w_active > 0)** — wait if anyone is active.

Always use a **while loop**, never an if-statement — threads can be woken spuriously!

◆ ■ Exam: Identifying Reader vs Writer Preference

Look at the unlock function. In `rwlock_writeunlock()`:

- If it checks **r_wait first** → broadcasts to readers → **Reader preference**
- If it checks **w_wait first** → signals writers → **Writer preference**

The example from the slides checks **r_wait** first → it prefers READERS.

■ 6. BARRIERS

What is a Barrier?

A **barrier** is a synchronization point: all threads in a group must reach it before any of them are allowed to continue past it. Think of it like a relay race baton exchange — everyone waits until the whole team is ready.

Visualizing a Barrier (5 threads)

Step	T1	T2	T3	T4	T5	Counter
Start	running	running	running	running	running	0
T1 arrives	■ wait	running	running	running	running	1
T1,T3,T4 waiting	■	running	■	■	running	3
All 5 arrived!	■	■	■	■	■	5 → WAKE UP!
After barrier	running	running	running	running	running	0 (reset)

When to Use Barriers

- Work is done in **phases** (e.g., step 1 must finish before step 2 begins)
- You know the number of worker threads **ahead of time**
- Classic examples: parallel matrix multiplication, image processing by rows, simulations

■ 7. POSIX BARRIER API

Function	What it does
<code>pthread_barrier_init(&b;, NULL, count)</code>	Initialize barrier. count = how many threads must call wait() before any proceed
<code>pthread_barrier_destroy(&b;)</code>	Clean up barrier when done
<code>pthread_barrier_wait(&b;)</code>	Block until count threads have called wait(). One thread gets PTHREAD_BARRIER_SERIAL_THREAD as return value, others get 0

Complete Barrier Example

```
#include

#include

#define NUM_THREADS 5

pthread_barrier_t barrier;

void* worker(void* arg) {

    int id = *(int*)arg;
```

```

printf("T%d: doing phase 1 work\n", id);

// ... phase 1 work here ...

// Wait for all threads to finish phase 1
int rc = pthread_barrier_wait(&barrier);

if (rc == PTHREAD_BARRIER_SERIAL_THREAD) {
    printf("I am the designated thread – can do serial cleanup\n");
}

// All threads continue from here together
printf("T%d: doing phase 2 work\n", id);

return NULL;
}

int main() {
    pthread_barrier_init(&barrier;, NULL, NUM_THREADS); // count = 5

    pthread_t threads[NUM_THREADS];

    int ids[NUM_THREADS];

    for (int i = 0; i < NUM_THREADS; i++) {
        ids[i] = i;

        pthread_create(&threads[i], NULL, worker, &ids[i]);
    }

    for (int i = 0; i < NUM_THREADS; i++)
        pthread_join(threads[i], NULL);

    pthread_barrier_destroy(&barrier);

    return 0;
}

```

◆ ■ PTHREAD_BARRIER_SERIAL_THREAD

Exactly **one** thread gets this return value from `pthread_barrier_wait()`. It can be used to designate one thread to do cleanup or setup work between phases. All other threads get 0. You don't control which thread is chosen.

■ 8. SIGNAL vs BROADCAST (Condition Variables Review)

pthread_cond_signal(&cond;)

- Wakes up **exactly ONE** thread waiting on the condition
- Use when only one thread can proceed (e.g., exactly one writer slot available)

```
// Wake one writer:

pthread_cond_signal(&write;_cv);
```

pthread_cond_broadcast(&cond;)

- Wakes up **ALL threads** waiting on the condition
- Use when multiple threads may proceed (e.g., all readers can now read)

```
// Wake all readers:

pthread_cond_broadcast(&read;_cv);
```

◆ ■■ Always Use while, Not if

After pthread_cond_wait() returns, re-check your predicate with a while loop. A thread can be woken spuriously (for no reason) or the condition may no longer be true by the time it acquires the mutex. while (condition) { pthread_cond_wait(...); } ← CORRECT

■ 9. ALLOWED vs NOT ALLOWED SYNCHRONIZATION

In this course, only **blocking** synchronization mechanisms are permitted.

■ ALLOWED	■ NOT ALLOWED
pthread_mutex_lock / unlock	while (flag == 0) {} ← busy-wait / spin lock
pthread_cond_wait / signal / broadcast	sleep(n) or usleep(n)
pthread_rwlock_rdlock / wrlock / unlock	nanosleep()
pthread_barrier_wait	Any polling / spinning loop for sync

◆ Why No Busy-Waiting?

Busy-waiting (spinning) wastes CPU — the thread keeps running and checking a variable instead of sleeping. Blocking functions release the CPU to other threads while waiting, which is far more efficient.

■ 10. QUICK REFERENCE — WHICH TOOL TO USE?

Situation	Use This	Key Function
Only one thread should access at a time	Mutex	pthread_mutex_lock/unlock
Thread must wait for a condition to become true	Mutex + Condition Variable	pthread_cond_wait/signal/broadcast
Many readers, occasional writers, read >> write	RW Lock	pthread_rwlock_rdlock/wrlock/unlock
All threads must complete phase N before phase N+1	Barrier	pthread_barrier_wait

Threads may deadlock (locking multiple mutexes)

**Fixed Order OR
Try-and-Back-Off**

Always lock in same order, or use
`trylock`

■ 11. COMMON EXAM TRAPS & TIPS

■ TRAP 1: Using `if` instead of `while` with `cond_wait`

→ Always use `while(predicate) { cond_wait(...); }` — spurious wakeups are real.

■ TRAP 2: Forgetting to unlock inside error paths

→ If you return early from a function that holds a lock, you **MUST** unlock first.

■ TRAP 3: Wrong lock type (read vs write)

→ Only use `rdlock` for purely read-only operations. Any modification needs `wrlock`.

■ TRAP 4: Mixing up signal and broadcast

→ `signal` wakes ONE thread; `broadcast` wakes ALL. Using `signal` when `broadcast` is needed causes starvation.

■ TRAP 5: Assuming `pthread_rwlock` is fair

→ POSIX `pthread_rwlock` prefers writers (blocks new readers when writers are waiting). Know which starvation that causes.

■ TRAP 6: Forgetting barrier count must match thread count

→ If `count=5` but only 4 threads call `wait()`, the barrier never releases — all threads block forever.

■ TRAP 7: Confusing deadlock and starvation

→ Deadlock = complete halt (circular waiting). Starvation = some threads make progress but others never get a turn.